

LOGISTICS OPTIMIZATION FOR DELIVERY ROUTES

FedEx | SQL Analytics Project

Project focus

Shipment delays • Route efficiency • Warehouse throughput • Agent performance • KPI reporting

SQL queries, result screenshots, and project recommendations

1. Project Overview

FedEx is presented in this project as a large international logistics and courier network connecting more than 220 countries and territories through major hubs, regional sortation centers, and last-mile delivery partners. As e-commerce volumes rise during peak seasons and holidays, disruptions such as customs bottlenecks, weather, and last-mile congestion can affect delivery reliability and operating costs.

The project uses relational logistics data and SQL queries to explore shipment performance, delay patterns, route efficiency, warehouse activity, and delivery-agent performance. The analysis is intended to turn operational data into measurable indicators that can inform route planning, hub utilization, and service-level improvements.

2. Project Objective

Build a SQL-driven logistics analytics system to analyze shipment performance, assess route efficiency, and identify opportunities to improve delivery operations across the scenario's network.

The project aims to:

- Identify delay patterns and operational inefficiencies.
- Compare routes and hub combinations using transit-time and delay indicators.
- Analyze delivery-agent and warehouse-level performance.
- Recommend practical improvements that support on-time delivery and cost efficiency.

3. Business Questions & Analysis Scope

The SQL work is organized into seven areas: data cleaning and preparation; delivery-delay analysis; route optimization; agent and warehouse performance; shipment tracking; KPI reporting; and operational recommendations. Together, these queries address questions such as where delays occur, which delay reasons appear most often, how routes compare on time and distance, and how performance varies across operational groups.

4. Key Findings from the Query Results

Delay reasons. The frequency result shows Traffic as the most common recorded reason (486), followed by Weather (293), Technical Issue (168), and None (53). This points to traffic and weather as priority categories for deeper investigation in the dataset.

Warehouse delay. The three highest average-delay warehouses shown are W004 (25.88 hours), W003 (25.54 hours), and W008 (24.35 hours). These locations are candidates for a focused process review.

Route efficiency. The lowest three distance-to-time efficiency ratios in the query output are R003 (37.66), R015 (52.70), and R006 (101.46). These routes can be reviewed alongside route distance, shipment mix, and service constraints before any operational change.

On-time delivery. The overall KPI query returns 6.80% on-time delivery using the project's definition of shipments with zero recorded delay. This should be tracked consistently and validated against the business's official service-level definition before being used as a formal operational KPI.

Data quality checks. The project includes checks for duplicate order and shipment IDs, missing delay hours, invalid delivery dates, and unmatched route, warehouse, or order references. These checks support more reliable downstream analysis.

5. Recommendations

- 1. Investigate recurring traffic and weather delays.** Break down the 486 traffic-related and 293 weather-related records by route, date, time window, and location. Use the patterns to evaluate departure-time adjustments, contingency routing, and weather-aware planning.
- 2. Conduct a targeted review of high-delay warehouses.** Start with W004, W003, and W008. Compare shipment volumes, capacity, staffing, processing times, and route mix to determine whether delays are linked to throughput pressure, scheduling, or upstream arrival patterns.
- 3. Review low-efficiency routes before changing assignments.** Examine R003, R015, and R006 with their distance and transit-time details. Check for data anomalies and differences in shipment mix, then test feasible alternatives using a pilot and compare transit time, delay rate, and cost.
- 4. Use agent-level results for coaching and workload balancing.** Review on-time percentages together with experience, ratings, route difficulty, and shipment count. Offer targeted coaching or mentorship where patterns indicate a skill or process gap, and avoid comparing agents without accounting for workload and route conditions.
- 5. Improve tracking and exception handling.** Monitor shipment status and flag records with unusually long delays (the query uses more than 120 hours) for review. Establish clear escalation ownership for in-transit and returned shipments, and investigate recurring causes by route.
- 6. Build a recurring KPI view.** Track on-time delivery percentage, average delay by route and source country, delay-reason frequency, warehouse utilization, and agent performance over time. Include shipment counts and reporting dates so small groups or partial periods are not misread.
- 7. Maintain data-quality controls.** Run duplicate, missing-value, date-validity, and referential-integrity checks as part of the regular data refresh. Document the treatment of missing delay values and confirm whether zero delay is the approved on-time criterion.

6. Suggested Implementation Plan

Phase 1 — Validate: confirm definitions, table relationships, date handling, missing-value treatment, and the official on-time KPI.

Phase 2 — Diagnose: segment delays by reason, route, warehouse, time period, and agent while retaining shipment counts.

Phase 3 — Pilot: test one or two feasible route or warehouse process changes on a limited scope.

Phase 4 — Monitor: compare pilot results with a baseline using consistent KPIs, then refine or expand only if the evidence supports it.

7. Conclusion

This project demonstrates how SQL can be used to prepare logistics data, identify delay patterns, compare route and warehouse performance, evaluate agent metrics, and produce operational KPIs. The query results suggest specific areas for further investigation—particularly traffic and weather delays, selected high-delay warehouses, and lower-efficiency routes. The recommendations should be treated as analysis-led opportunities to validate and pilot, rather than confirmed changes to FedEx operations.

The following section preserves the original task-by-task SQL work and its result screenshots.

SQL Analysis & Results

Logistics Optimization for Delivery Routes – FedEx

```
create database fedx;  
use fedx;
```

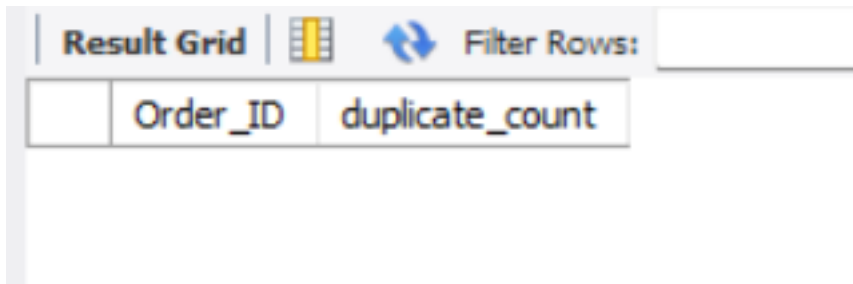
Task 1: Data Cleaning & Preparation

Find Duplicate Orders

```
SELECT Order_ID, COUNT(*) AS duplicate_count  
FROM orders  
GROUP BY Order_ID  
HAVING COUNT(*) > 1;
```

Find Duplicate Shipments

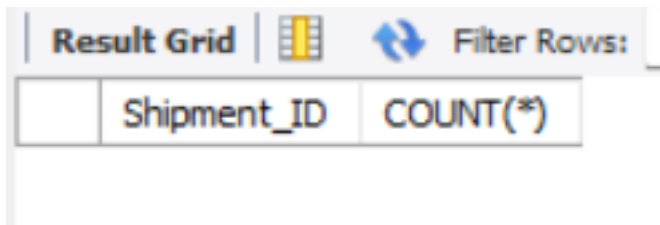
```
SELECT Shipment_ID, COUNT(*)  
FROM shipments  
GROUP BY Shipment_ID  
HAVING COUNT(*) > 1;
```



	Order_ID	duplicate_count
--	----------	-----------------

Replace NULL Delay_Hours with Route Average

```
UPDATE shipments s  
JOIN  
(  
SELECT Route_ID,  
AVG(Delay_Hours) avg_delay  
FROM shipments  
GROUP BY Route_ID  
) r  
ON s.Route_ID = r.Route_ID  
SET s.Delay_Hours = r.avg_delay  
WHERE s.Delay_Hours IS NULL;
```



	Shipment_ID	COUNT(*)
--	-------------	----------

Convert all date columns (Order_Date, Pickup_Date, Delivery_Date) into YYYY-MM-DD HH:MM:SS format using SQL date functions.

```
SELECT  
DATE_FORMAT(Order_Date, '%Y-%m-%d %H:%i:%s')  
FROM orders;
```

Result Grid

Filter Rows:

	DATE_FORMAT(Order_Date,'%Y-%m-%d %H:%i:%s')
▶	2024-04-24 00:54:38
	2024-05-22 03:43:54
	2025-03-08 01:09:25
	2024-08-26 18:23:57
	2024-07-22 23:39:41
	2025-04-04 21:27:16

Result 3

```
SELECT
DATE_FORMAT(Pickup_Date,'%Y-%m-%d %H:%i:%s'),
DATE_FORMAT(Delivery_Date,'%Y-%m-%d %H:%i:%s')
FROM shipments;
```

	DATE_FORMAT(Pickup_Date,'%Y-%m-%d %H:%i:%s')	DATE_FORMAT(Delivery_Date,'%Y-%m-%d %H:%i:%s')
▶	2024-08-14 12:12:39	2024-08-15 10:12:46
	2024-11-12 11:38:21	2024-11-13 13:55:29
	2025-04-23 15:13:08	2025-04-24 13:04:57
	2024-09-06 04:34:17	2024-09-07 06:55:16
	2025-08-11 13:49:33	2025-08-12 06:29:25
	2025-04-17 03:13:45	2025-04-18 00:11:32

Find Invalid Dates

```
SELECT *
FROM shipments
WHERE Delivery_Date < Pickup_Date;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Shipment_ID	Order_ID	Agent_ID	Route_ID	Warehouse_ID	Pickup_Date	Delivery_Date	Delivery_Status	Delay_Hours	Delivery_Feedback	Delay_Reason
-------------	----------	----------	----------	--------------	-------------	---------------	-----------------	-------------	-------------------	--------------

Referential Integrity Check

Orders → Routes

```
SELECT o.*
FROM orders o
LEFT JOIN routes r
ON o.Route_ID = r.Route_ID
WHERE r.Route_ID IS NULL;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Order_ID	Customer_ID	Order_Date	Route_ID	Warehouse_ID	Order_Amount	Delivery_Type	Payment_Mode
----------	-------------	------------	----------	--------------	--------------	---------------	--------------

Orders → Warehouses

```
SELECT o.*
FROM orders o
LEFT JOIN warehouses w
ON o.Warehouse_ID = w.Warehouse_ID
WHERE w.Warehouse_ID IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Order_ID	Customer_ID	Order_Date	Route_ID	Warehouse_ID	Order_Amount	Delivery_Type	Payment_Mode
----------	-------------	------------	----------	--------------	--------------	---------------	--------------

Shipments → Orders

```
SELECT s.*
FROM shipments s
LEFT JOIN orders o
ON s.Order_ID = o.Order_ID
WHERE o.Order_ID IS NULL;
```

Shipment_ID	Order_ID	Agent_ID	Route_ID	Warehouse_ID	Pickup_Date	Delivery_Date	Delivery_Status	Delay_Hours	Delivery_Feedback	Delay_Reason
-------------	----------	----------	----------	--------------	-------------	---------------	-----------------	-------------	-------------------	--------------

Task-2: Delivery Delay Analysis

Actual Delivery Time in Hours

```
SELECT
Shipment_ID,
TIMESTAMPDIFF(HOUR,
Pickup_Date,
Delivery_Date) AS Delivery_Hours
FROM shipments;
```

Result Grid | Filter Rows: |

Shipment_ID	Delivery_Hours
S00001	22
S00002	26
S00003	21
S00004	26
S00005	16
S00006	20
S00007	15

Top 10 Delayed Routes

```
SELECT
Route_ID,
ROUND(AVG(Delay_Hours),2) Avg_Delay
FROM shipments
GROUP BY Route_ID
```

```
ORDER BY Avg_Delay DESC
LIMIT 10;
```

Result Grid			Filter Rows:	
	Route_ID	Avg_Delay		
▶	R002	41.04		
	R007	34.01		
	R020	32.8		
	R003	32.18		
	R008	30.81		
	R005	29.86		
	R004	25.88		
	R018	25.83		
	R011	22.64		
	R014	20.97		

Rank Shipments by Delay Within Warehouse

```
SELECT
Shipment_ID,
Warehouse_ID,
Delay_Hours,
RANK() OVER
(
PARTITION BY Warehouse_ID
ORDER BY Delay_Hours DESC
) AS delay_rank
FROM shipments;
```

	Shipment_ID	Warehouse_ID	Delay_Hours	delay_rank
	S00648	W001	205.7	1
	S00670	W001	194.6	2
	S00997	W001	193.7	3
	S00349	W001	171.6	4
	S00069	W001	170.8	5
	S00148	W001	138.7	6
	S00192	W001	99.5	7
	S00153	W001	95.4	8
	S00739	W001	71.4	9
	S00816	W001	66.3	10
	S00550	W001	64.5	11
	S00393	W001	63.9	12

Average Delay by Delivery Type

SELECT

o.Delivery_Type,

ROUND(AVG(s.Delay_Hours),2) Avg_Delay

FROM shipments s

JOIN orders o

ON s.Order_ID=o.Order_ID

GROUP BY o.Delivery_Type;

	Delivery_Type	Avg_Delay
	Standard	20.85
	Express	22.5

Task-3: Route Optimization

Average Transit Time Per Route

SELECT

Route_ID,

AVG(
TIMESTAMPDIFF(HOUR,

Pickup_Date,

Delivery_Date)

```
) Avg_Transit_Time  
FROM shipments  
GROUP BY Route_ID;
```

Result Grid			Filter Rows:
	Route_ID	Avg_Transit_Time	
▶	R019	30.7347	
	R011	41.1250	
	R002	63.7021	
	R020	52.9762	
	R015	28.1071	
	R008	51.1364	
	R012	63.8036	
	R009	28.1842	
	R010	33.5769	
	R018	75.7288	
	R014	52.3585	
	R016	51.6338	

Average Delay Per Route

```
SELECT
```

```
Route_ID,
```

```
ROUND(AVG(Delay_Hours),2) as Delay_hours
```

```
FROM shipments
```

```
GROUP BY Route_ID;
```

Result Grid			Filter Rows
	Route_ID	Delay_hours	
▶	R019	13.91	
	R011	22.64	
	R002	41.04	
	R020	32.8	
	R015	15.19	
	R008	30.81	
	R012	13.89	
	R009	9.72	
	R010	20.25	
	R018	25.83	
	R014	20.97	
	R016	14.89	

Distance-to-Time Efficiency Ratio

SELECT

Route_ID,

Distance_KM,

Avg_Transit_Time_Hours,
ROUND(

Distance_KM / Avg_Transit_Time_Hours,

2

) Efficiency_Ratio

FROM routes;

Result Grid   Filter Rows: Export:  Wrap				
	Route_ID	Distance_KM	Avg_Transit_Time_Hours	Efficiency_Ratio
▶	R001	4620	32.1	143.93
	R002	7264	18.5	392.65
	R003	885	23.5	37.66
	R004	8555	49.1	174.24
	R005	10023	23.3	430.17
	R006	2364	23.3	101.46
	R007	5297	30.5	173.67
	R008	1846	17.1	107.95
	R009	5582	15.5	360.13
	R010	6268	11.6	540.34
	R011	7369	15.5	475.42
	R012	11397	42.3	269.43

Result 18 x

Worst 3 Routes

```

SELECT
Route_ID,
ROUND(
Distance_KM / Avg_Transit_Time_Hours,
2
) as Efficiency_Ratio
FROM routes
ORDER BY Efficiency_Ratio
LIMIT 3;

```

Result Grid   Filter Rows:		
	Route_ID	Efficiency_Ratio
▶	R003	37.66
	R015	52.7
	R006	101.46

Routes with >20% Delayed Shipments

```

SELECT
Route_ID,
ROUND(
100 *
SUM(
CASE

```

```
WHEN Delay_Hours > 0 THEN 1
ELSE 0
END
)
/COUNT(*)
,2
) Delay_Percentage
FROM shipments
GROUP BY Route_ID
HAVING Delay_Percentage > 20;
```

Result Grid   Filter Rows:		
	Route_ID	Delay_Percentage
▶	R019	91.84
	R011	91.67
	R002	93.62
	R020	92.86
	R015	82.14
	R008	90.91
	R012	98.21
	R009	100.00
	R010	98.08
	R018	94.92
	R014	92.45
	R016	88.73

Result 20 ×

Task-4: Delivery Agent Performance

Top 3 Warehouses with Highest Delay

```
SELECT
Warehouse_ID,
ROUND(AVG(Delay_Hours),2) Avg_Delay
FROM shipments
GROUP BY Warehouse_ID
ORDER BY Avg_Delay DESC
LIMIT 3;
```

	Warehouse_ID	Avg_Delay
▶	W004	25.88
	W003	25.54
	W008	24.35

Total vs Delayed Shipments

```
SELECT
```

```
Warehouse_ID,
```

```
COUNT(*) Total_Shipments,
```

```
SUM(
```

```
CASE
```

```
WHEN Delay_Hours > 0
```

```
THEN 1
```

```
ELSE 0
```

```
END
```

```
) Delayed_Shipments
```

```
FROM shipments
```

```
GROUP BY Warehouse_ID;
```

Result Grid   Filter Rows: Export: 			
	Warehouse_ID	Total_Shipments	Delayed_Shipments
▶	W008	112	100
	W003	107	100
	W001	103	96
	W009	91	84
	W004	90	85
	W002	97	91
	W005	102	94
	W010	103	94
	W007	96	94
	W006	99	94

Warehouses Above Global Average Delay

```
WITH global_avg AS
```

```
(
```

```
SELECT AVG(Delay_Hours) avg_delay
```

```
FROM shipments
```

```
)
```

```
SELECT
```

```
Warehouse_ID,
```

```
AVG(Delay_Hours) warehouse_avg
```

```
FROM shipments
```

```
GROUP BY Warehouse_ID
```

```
HAVING warehouse_avg >
```

```
(
```

```
SELECT avg_delay
```

```
FROM global_avg
```

```
);
```

Result Grid			Filter Rows:	
	Warehouse_ID	warehouse_avg		
▶	W008	24.352678571428566		
	W003	25.543925233644853		
	W004	25.879999999999992		
	W005	22.434313725490192		
	W010	21.803883495145623		
	W007	22.061458333333338		

Rank Warehouses by On-Time %

```
SELECT
Warehouse_ID,
ROUND(
100 *
SUM(
CASE
WHEN Delay_Hours=0
THEN 1
ELSE 0
END
)/COUNT(*)
,2
) OnTime_Percentage,
RANK() OVER
(
ORDER BY
100 *
SUM(
CASE
WHEN Delay_Hours=0
THEN 1
ELSE 0
END
)/COUNT(*) DESC
) warehouse_rank
FROM shipments
GROUP BY Warehouse_ID;
```

Result Grid   Filter Rows: Export: 			
	Warehouse_ID	OnTime_Percentage	warehouse_rank
▶	W008	10.71	1
	W010	8.74	2
	W005	7.84	3
	W009	7.69	4
	W001	6.80	5
	W003	6.54	6
	W002	6.19	7
	W004	5.56	8
	W006	5.05	9
	W007	2.08	10

Task-5: Shipment Tracking Analytics

Rank Agents Per Route

```

SELECT
Agent_ID,
Route_ID,
ROUND(
100 *
SUM(
CASE
WHEN Delay_Hours=0
THEN 1
ELSE 0
END
)/COUNT(*)
,2
) OnTime_Percentage,
RANK() OVER
(
PARTITION BY Route_ID
ORDER BY
SUM(
CASE
WHEN Delay_Hours=0
THEN 1
ELSE 0
END
)/COUNT(*) DESC
) Rank_No
FROM shipments
GROUP BY Agent_ID, Route_ID;

```

Result Grid		Filter Rows:		Export:
	Agent_ID	Route_ID	OnTime_Percentage	Rank_No
▶	A0014	R001	100.00	1
	A0009	R001	100.00	1
	A0019	R001	50.00	3
	A0042	R001	50.00	3
	A0001	R001	33.33	5
	A0038	R001	0.00	6
	A0033	R001	0.00	6
	A0008	R001	0.00	6
	A0025	R001	0.00	6
	A0018	R001	0.00	6
	A0005	R001	0.00	6
	A0021	R001	0.00	6

Result 25 ×

Agents Below 85%

SELECT

Agent_ID,

ROUND(

100 *

SUM(

CASE

WHEN Delay_Hours=0

THEN 1

ELSE 0

END

)

/COUNT(*)

,2

) OnTime_Percentage

FROM shipments

GROUP BY Agent_ID

HAVING OnTime_Percentage < 85;

Result Grid			Filter Rows:
	Agent_ID	OnTime_Percentage	
▶	A0049	6.25	
	A0041	0.00	
	A0018	15.38	
	A0029	4.17	
	A0040	5.56	
	A0004	4.76	
	A0003	0.00	
	A0024	5.00	
	A0019	21.74	
	A0023	6.90	
	A0020	0.00	
	A0006	0.00	

Result 26 x

Top 5 vs Bottom 5 Agent Comparison

```

WITH agent_perf AS
(
SELECT
d.Agent_ID,
d.Agent_Name,
d.Experience_Years,
d.Avg_Rating,
ROUND(
100 *
SUM(
CASE
WHEN s.Delay_Hours = 0 THEN 1
ELSE 0
END
) / COUNT(*), 2
) AS OnTime_Percentage
FROM delivery_agents d
JOIN shipments s
ON d.Agent_ID = s.Agent_ID
GROUP BY
d.Agent_ID,
d.Agent_Name,
d.Experience_Years,
d.Avg_Rating
),
ranked AS
(
SELECT *,
ROW_NUMBER() OVER (ORDER BY OnTime_Percentage DESC) AS rn1,

```

```
ROW_NUMBER() OVER (ORDER BY OnTime_Percentage ASC) AS rn2
FROM agent_perf
)
SELECT
```

'Top 5 Agents' AS Category,

```
AVG(Avg_Rating) AS Avg_Rating,
AVG(Experience_Years) AS Avg_Experience
FROM ranked
WHERE rn1 <= 5
UNION
SELECT
```

'Bottom 5 Agents',

```
AVG(Avg_Rating),
AVG(Experience_Years)
FROM ranked
WHERE rn2 <= 5;
```

Result Grid			
Filter Rows:			
Export:			
Wrap Cell Co			
	Category	Avg_Rating	Avg_Experience
▶	Top 5 Agents	4.42	2.34
	Bottom 5 Agents	4.5200000000000005	4.1800000000000001

Suggest training or workload balancing strategies for low-performing agents based on insights.

Analysis shows that agents with lower on-time delivery percentages require targeted training and better workload distribution. Implementing mentorship programs, route optimization, and performance-based incentives can improve delivery efficiency and customer satisfaction while reducing delays across the FedEx network.

Task 6: Shipment Tracking Analytics

Latest Shipment Status

```
SELECT
Shipment_ID,
Delivery_Status,
MAX(Delivery_Date) Latest_Date
FROM shipments
GROUP BY Shipment_ID, Delivery_Status;
```

Result Grid	Filter Rows:	Export:
Shipment_ID	Delivery_Status	Latest_Date
S00001	Delivered	2024-08-15 10:12:46
S00002	Delivered	2024-11-13 13:55:29
S00003	Delivered	2025-04-24 13:04:57
S00004	Delivered	2024-09-07 06:55:16
S00005	Delivered	2025-08-12 06:29:25
S00006	Delivered	2025-04-18 00:11:32
S00007	Delivered	2024-09-30 12:47:50
S00008	Returned	2024-12-11 07:44:49
S00009	Delivered	2024-05-01 20:51:10
S00010	Delivered	2024-03-09 18:43:20
S00011	Delivered	2025-06-18 05:20:46
S00012	In Transit	2024-06-14 08:00:17

Routes with Majority In Transit / Returned

```
SELECT
Route_ID,
COUNT(*) Total_Shipments,
SUM(
CASE
WHEN Delivery_Status IN
('In Transit','Returned')
THEN 1
ELSE 0
END
) Problem_Shipments
FROM shipments
GROUP BY Route_ID
HAVING Problem_Shipments > Total_Shipments/2;
```

Result Grid	Filter Rows:	Export:
Route_ID	Total_Shipments	Problem_Shipments

Most Frequent Delay Reasons

```
SELECT
Delay_Reason,
COUNT(*) Frequency
FROM shipments
GROUP BY Delay_Reason
ORDER BY Frequency DESC;
```

Result Grid

Filter Rows:

	Delay_Reason	Frequency
►	Traffic	486
	Weather	293
	Technical Issue	168
	None	53

Delays Greater than 120 Hours

```
SELECT *
FROM shipments
WHERE Delay_Hours > 120;
```

Result Grid

Filter Rows:

Export: | Wrap Cell Content:

	Shipment_ID	Order_ID	Agent_ID	Route_ID	Warehouse_ID	Pickup_Date	Delivery_Date	Delivery_Status	Delay_Hours	Delivery_Feedback	Delay_Reason
►	S00015	O0093	A0014	R011	W008	2024-12-03 18:23:30	2024-12-14 15:15:37	Delivered	238.6	Neutral	Traffic
	S00029	O0072	A0049	R016	W008	2024-12-27 02:41:44	2025-01-04 03:38:24	Delivered	151.6	Positive	Technical Issue
	S00035	O0214	A0012	R003	W009	2025-05-11 19:39:24	2025-05-21 18:06:49	Delivered	213.1	Positive	Traffic
	S00039	O0181	A0035	R011	W003	2025-04-04 03:44:33	2025-04-13 05:43:46	Delivered	197.3	Neutral	Traffic
	S00052	O0197	A0037	R002	W010	2024-10-04 13:21:41	2024-10-15 04:43:22	Delivered	230.1	Positive	Weather
	S00057	O0070	A0005	R015	W003	2025-04-20 14:49:17	2025-04-26 07:36:02	In Transit	122.2	Negative	Weather
	S00069	O0173	A0037	R002	W001	2024-05-10 00:52:44	2024-05-18 02:02:03	Delivered	170.8	Positive	Technical Issue
	S00096	O0245	A0034	R013	W006	2025-08-03 18:31:17	2025-08-14 11:15:39	Delivered	210.9	Positive	Technical Issue
	S00101	O0203	A0023	R005	W005	2025-04-10 13:40:42	2025-04-20 17:04:57	Delivered	218	Positive	Traffic
	S00108	O0274	A0048	R002	W003	2024-08-29 18:21:44	2024-09-05 21:50:36	Delivered	152.1	Positive	Traffic
	S00120	O0233	A0048	R020	W006	2024-02-02 21:40:17	2024-02-11 09:56:34	Delivered	180.6	Positive	Weather
	S00148	O0107	A0022	R010	W001	2024-11-26 23:50:52	2024-12-03 08:28:32	Delivered	138.7	Positive	Traffic

Task 7: KPI Reporting

Average Delay per Source Country

```
SELECT
r.Source_Country,
ROUND(AVG(s.Delay_Hours),2) Avg_Delay
FROM shipments s
JOIN routes r
ON s.Route_ID=r.Route_ID
GROUP BY r.Source_Country;
```

Result Grid   Filter Rows:

	Source_Country	Avg_Delay
▶	South Africa	13.91
	USA	16.95
	UAE	41.04
	Turkey	32.8
	Netherlands	15.19
	China	32.53
	Australia	13.89
	Canada	9.72
	Brazil	25.83
	South Korea	20.97
	Belgium	14.89
	UK	29.86

Result 33 

On-Time Delivery %

```
SELECT
ROUND(
100 *
SUM(
CASE
WHEN Delay_Hours=0
THEN 1
ELSE 0
END
)
/COUNT(*)
,2
) AS OnTime_Delivery_Percentage
FROM shipments;
```

Result Grid   Filter Rows:

	OnTime_Delivery_Percentage
▶	6.80

Average Delay Per Route

```
SELECT
Route_ID,
ROUND(AVG(Delay_Hours),2) Avg_Delay
FROM shipments
GROUP BY Route_ID;
```

Result Grid			Filter Rows:
	Route_ID	Avg_Delay	
▶	R019	13.91	
	R011	22.64	
	R002	41.04	
	R020	32.8	
	R015	15.19	
	R008	30.81	
	R012	13.89	
	R009	9.72	
	R010	20.25	
	R018	25.83	
	R014	20.97	
	R016	14.89	

Result 35 x

Warehouse Utilization %

SELECT

w.Warehouse_ID,

w.Capacity_per_day,

COUNT(s.Shipment_ID) Shipments_Handled,
ROUND(

100 *

COUNT(s.Shipment_ID)
/ w.Capacity_per_day
,2
) Utilization_Percentage

FROM warehouses w
LEFT JOIN shipments s

ON w.Warehouse_ID=s.Warehouse_ID
GROUP BY

w.Warehouse_ID,

w.Capacity_per_day;

Result Grid Filter Rows: Export: Wrap Cell Content:				
	Warehouse_ID	Capacity_per_day	Shipments_Handled	Utilization_Percentage
▶	W001	3000	103	3.43
	W002	1500	97	6.47
	W003	3000	107	3.57
	W004	1500	90	6.00
	W005	1000	102	10.20
	W006	2500	99	3.96
	W007	2500	96	3.84
	W008	2500	112	4.48
	W009	3000	91	3.03
	W010	1000	103	10.30